

CSPG Index in FAISS

Dimitra Seferiadi
AM: 03120131

Abstract

This report describes an implementation of the Crossing Sparse Proximity Graph (CSPG) index [1] within Facebook AI Similarity Search (FAISS) [2]. CSPG, proposed by Yang, Cai, and Zheng (NeurIPS 2024), replaces the single monolithic proximity graph of mainstream graph-based Approximate Nearest Neighbor (ANN) indexes with m *sparser* proximity graphs built on random partitions of the dataset that share a common set of *routing vectors*. A two-stage search, a fast-approaching stage on one partition followed by a cross-partition expansion over all graphs through the routing vectors, is argued to reduce the expected number of distance computations compared to a single graph on the whole dataset. We implement CSPG as a native FAISS index class (`IndexCSPG`) whose partitions are instances of `IndexHNSWFlat` and whose two-stage search is built on top of FAISS’s `MinimaxHeap`, batch-4 L2 distances, and per-thread `VisitedTables`. We evaluate the implementation on the full CSPG paper suite plus UQV1M and OpenAI1M, covering dimensionalities from 96 to 1536 and scales from 1M to 10M, together with a SIFT{0.1, 0.2, 0.5, 2, 5}M scaling series for the n -varying and detour studies. Relative to the paper’s reported 1.5–1.8 $\times$ QPS speedup against HNSWlib, our CSPG implementation lands in a more nuanced regime against FAISS’s tightly SIMD-optimised HNSW: it is slower at matched recall across all six fixed- n datasets, but it reaches higher peak recall ceilings on UQV1M (+0.9 pp at $k=10$) and OpenAI1M (0.9990 vs 0.9983 at $k=10$), substantially improves tail-query robustness on OpenAI1M (mean recall@20 0.9938 vs 0.9907, worst-case min 0.70 vs 0.65, a +5 pp tail win), delivers a +5 pp worst-case win on SIFT10M, and adds only +2.5% memory overhead at $d = 1536$. The wins cluster exactly in the regime CSPG’s theory predicts - high dimensionality and hard-tail queries - confirming the algorithm’s direction of improvement while exposing that the per-hop overhead of the encoded (p, lid) heap and cross-partition broadcast becomes the bottleneck when FAISS’s batch-4 SIMD distance already saturates the pipe. Ablations on m and λ reproduce the paper’s qualitative trends (larger λ improves recall-per-distance; the optimal m depends on the dataset), and confirm that $\text{ef}_1=1$ is the best first-stage setting.

1 Introduction

Graph-based Approximate Nearest Neighbor (ANN) indexes, HNSW [3], Vamana, NSG, and HCNNG, are the standard in production vector databases because they combine logarithmic search complexity with high recall. All of them share the same structural template: *one* proximity graph is built on the entire dataset, and the query is answered by a best-first beam search seeded from a good entry point.

The dominant cost of that beam search is the number of distance computations performed: at each node, all of its neighbors are evaluated against the query, and promising ones are pushed into the candidate heap. The number of explored nodes can be bounded analytically for idealized graphs (MSNETs [4]), and empirically grows with the size and density of the graph. This suggests a natural question: *can we reduce the number of distance computations by searching a **smaller** graph without sacrificing recall?*

The Crossing Sparse Proximity Graph (CSPG) [1], answers yes by randomly partitioning the dataset into m smaller groups that *share* a sampled subset of *routing vectors*, and building an

independent proximity graph per partition. Because each partition has fewer vectors, its graph is sparser and cheaper to traverse. The routing vectors are present in all partitions and serve as “gateways” that let a greedy beam search cross between graphs at query time. The paper proves that, under the assumption that each partition’s graph is an Approximate Monotonic Search Network, CSPG strictly outperforms the single PG baseline in expected distance computations, and experimentally reports 1.5–2.0× QPS speedups.

2 Paper Overview: The CSPG Algorithm

2.1 Problem Setting and Graph-Based ANN

Let $\mathcal{D} = \{v_1, \dots, v_n\} \subset \mathbb{R}^d$ be a dataset under the Euclidean distance $\delta(\cdot, \cdot)$. The *k-Approximate Nearest Neighbor Search* (*k*-ANNS) problem asks, given a query $q \in \mathbb{R}^d$, for a set $S \subseteq \mathcal{D}$ of size k with $|S \cap T|$ as large as possible, where T is the set of true k nearest neighbors of q . Graph-based ANNS methods construct a proximity graph $\text{PG}(\mathcal{D})$ on $\mathcal{D}$ and answer queries by best-first greedy beam search. The expected running cost is $\mathbb{E}[C] = \sigma \cdot \mathbb{E}[|p \rightsquigarrow q|]$, where σ is the average degree and $|p \rightsquigarrow q|$ is the length of the search sequence from the entry point p to the query q [4].

2.2 Random Partitioning and Routing Vectors

Definition 1 (Random partition, Definition 5). *Given $\mathcal{D}$, a random partition $\mathcal{P}_1, \dots, \mathcal{P}_m$ satisfies (i) $\bigcup_i \mathcal{P}_i = \mathcal{D}$, (ii) $\bigcap_i \mathcal{P}_i = \mathcal{C}$, and (iii) $(\mathcal{P}_i \setminus \mathcal{C}) \cap (\mathcal{P}_j \setminus \mathcal{C}) = \emptyset$ for $i \neq j$, where $\mathcal{C}$ is the set of vectors shared by all partitions (the routing vectors, $|\mathcal{C}| = \lambda n$), and $\mathcal{P}_i$ receives a uniformly random assignment of the non-routing vectors.*

Definition 2 (CSPG index, Definition 6). *Given $\mathcal{D}$ and a random partition $\{\mathcal{P}_i\}$, $\text{CSPG}(\mathcal{D}) = \{\mathcal{G}_1, \dots, \mathcal{G}_m\}$ where $\mathcal{G}_i = \text{PG}(\mathcal{P}_i)$.*

Because each $\mathcal{P}_i$ contains $\lambda n + (1 - \lambda)n/m$ vectors (much smaller than n), each partition graph $\mathcal{G}_i$ is sparser than the monolithic graph $\text{PG}(\mathcal{D})$. Routing vectors are the critical ingredient: they appear in every partition graph, which means that the *same* physical vector has an incident edge in every $\mathcal{G}_i$, and can therefore serve as a bridge that lets a greedy search step from $\mathcal{G}_i$ directly into $\mathcal{G}_j$.

2.3 Two-Stage Search on CSPG (Algorithm 1)

The CSPG search procedure has two stages with *different* candidate-set sizes $\text{ef}_1 < \text{ef}_2$:

Stage 1 (fast approaching) Starting from an entry vector $p \in \mathcal{P}_1$, a greedy beam search of candidate size ef_1 is run on $\mathcal{G}_1$ alone. Because $\mathcal{G}_1$ is sparser, each hop is cheap; because ef_1 is small (ideally 1), the exploration is fast and quickly reaches the neighborhood of q .

Stage 2 (cross-partition expansion) Starting from the closest vector found in Stage 1, a second beam search with a larger candidate size ef_2 is run. The key step (line 13 of Algorithm 1) fires whenever an *expanded* neighbor u is itself a routing vector: all of u ’s $(m - 1)$ replicas $\{(u, i) \mid i \neq \text{current partition}\}$ are inserted into the candidate heap. This lets the beam search step into every other partition’s graph *at that vertex*, without having to walk there through $\mathcal{G}_1$. The top- k result is returned from the visited set at the end.

2.4 Theoretical Advantage (Section 5)

Assuming each $\mathcal{G}_i$ is an Approximate Monotonic Search Network (AMSNET), the paper proves a speedup theorem:

$$\text{Speedup} = \frac{\mathbb{E}^{\text{PG}}[|p \rightsquigarrow q|]}{\mathbb{E}^{\text{CSPG}}[|p \rightsquigarrow q|]} = \alpha\beta > 1, \quad \beta = \frac{n^{1/d} \log n^{1/d}}{\left(\lambda n + \frac{n(1-\lambda)}{m}\right)^{1/d} \log\left(\lambda n + \frac{n(1-\lambda)}{m}\right)^{1/d}}$$

where $\alpha \geq 1$ accounts for graph sparsity (edges in $\mathcal{G}_i$ are shorter than in $\text{PG}(\mathcal{D})$) and $\beta > 1$ as long as each partition is strictly smaller than $\mathcal{D}$ ($m > 1$ and $\lambda < 1$). The speedup is expressed in *number of distance computations*, not in wall-clock time; any implementation that adds constant overhead per hop (heap encoding, indirection, cross-expansion bookkeeping) erodes this advantage.

2.5 Complexity

For a dataset of n vectors and a base PG index of time/space cost $T(n)/S(n)$, each partition has $\frac{n(1-\lambda)}{m} + \lambda n$ vectors, so CSPG’s build costs are $\mathcal{O}(m \cdot T(\frac{n(1-\lambda)}{m} + \lambda n))$ and $\mathcal{O}(m \cdot S(\frac{n(1-\lambda)}{m} + \lambda n))$. At $\lambda = 0.5, m = 2$ the total number of indexed points is $(1 - \lambda)n + \lambda nm = 1.5n$, so CSPG’s index size is roughly $1.5\times$ that of the baseline PG, the main cost CSPG pays for its search-time advantage.

3 Implementation: IndexCSPG in FAISS

Our implementation lives in `faiss/IndexCSPG.{h,cpp}` and `IndexCSPG_io.h`. The class inherits from `Index` and holds a single `IndexFlat*` `shared_flat` (all n raw vectors, stored *once*), a vector of `IndexHNSWFlat*` partitions (one HNSW per $\mathcal{P}_i$), and a two-level table `refunction[p][lid]` that maps each partition-local ID to its global ID in `shared_flat`. Routing vectors occupy local IDs $0 \dots \text{num_routing}-1$ in *every* partition, mapped to the same sorted sequence of global IDs, so the invariant “`lid < num_routing` $\iff$ vertex is a routing vector” drives the Stage-2 cross-partition broadcast without any auxiliary data structure. The `shared_flat` design avoids physically duplicating the λn routing vectors across partitions, a naive CSPG implementation would waste $(m-1)\lambda n d$ floats.

Construction (Algorithm 2) The `add(n, x)` method builds the index in four steps. First, it picks the $\lfloor \lambda n \rfloor$ routing vectors by shuffling all global IDs with a fixed seed and taking the prefix, this makes the choice uniform over the dataset (so input ordering does not bias which vectors become routes) and reproducible across runs. Second, every remaining ID is assigned to exactly one partition uniformly at random. Third, the raw vectors are copied once into `shared_flat`. Fourth, for each partition we gather its assigned vectors into a contiguous buffer and call FAISS’s `IndexHNSWFlat::add` on it, which reuses FAISS’s level assignment, bidirectional link insertion, and `efConstruction` heuristic without modification. Once each partition’s HNSW graph is built, we discard the partition’s internal `IndexFlat` copy of the vectors, search will read all vector data from `shared_flat` via the `refunction` table, so the duplicate is dead weight.

Adapter: `RemappedDistanceComputer` FAISS’s HNSW helpers (`greedy_update_nearest`, `search_from_candidate_unbounded`) expect distances in partition-local IDs, but our queries must hit `shared_flat` in global IDs. Rather than forking these helpers, we wrap the `shared_flat` `DistanceComputer` in a thin adapter that applies `mapping[i] = refunction[p][i]` before every scalar and batch-4 distance call. One adapter per thread is reused across partitions via `set_mapping()`, keeping the SIMD `set_query` primed for the whole per-query search.

Two-stage search (Algorithm 1) The candidate heap (`MinimaxHeap`) stores a single integer, so we pack (p, lid) as $\text{enc} = p \cdot \text{stride} + \text{lid}$ with $\text{stride} = \max_p \text{ntotal}_p$; encode/decode is one multiply/modulo per pop. Stage 1 runs FAISS’s top-down HNSW descent on partition 0 alone with $\text{ef}_1=1$ (ablation-confirmed optimum). Stage 2 maintains two heaps: a `candidates` `MinimaxHeap` oversized to $\text{ef}_2 \cdot m$ (so that cross-partition broadcasts do not evict good candidates), and a `top_results` max-heap of capacity ef_2 whose front serves as both termination threshold *and* crucially the filter that gates neighbor acceptance (an early version gated on `candidates.max()`, which was too loose and roughly doubled the hop count at matched recall).

Two design tricks matter. (i) *Eager seed broadcast*: if the Stage-1 winner is itself a routing vector, it is pushed into every partition’s candidate heap and marked visited there, sparing Stage 2 from re-walking $\mathcal{G}_1$ until it happens to re-discover it. (ii) *Cross-partition broadcast on accept*: whenever an accepted neighbor is a routing vector, its replicas in the remaining $m-1$ partitions are pushed *and marked visited*, so the same global vertex can never enter `top_results` twice and runtime deduplication is avoided. The main loop processes neighbors in three passes (prefetch visited bits $\rightarrow$ collect unvisited + prefetch L2 vectors $\rightarrow$ `fvec.L2sqr_batch_4` in groups of four), the same SIMD pattern as `IndexHNSWFlat`. The per-query loop is wrapped in `#pragma omp parallel` with a per-thread `VisitedTable` array (one per partition, $O(1)$ reset via generation counter) and dynamic scheduling.

Serialisation Serialisation uses FAISS’s `IOWriter/IOWriter` macros, so `IndexCSPG` round-trips through the standard `faiss::write_index/read_index` interface.

Reuse vs fork We prefer to inherit from existing FAISS components rather than fork them whenever possible: graph construction, SIMD distance kernels, `MinimaxHeap`, `VisitedTable`, and I/O macros are all reused unchanged. CSPG-specific code is confined to the `RemappedDistanceComputer` adapter, the encoded-heap two-stage search loop, and the I/O block. Table 1 summarises the component breakdown.

Table 1: FAISS infrastructure: reused, extended, or new in `IndexCSPG`.

Component	Status	Rationale
<code>IndexHNSWFlat</code> , HNSW struct	Reused (one per partition)	Graph topology + level navigation
<code>IndexFlat</code>	Reused (as <code>shared_flat</code>)	Single shared vector store
<code>MinimaxHeap</code>	Reused	Stage-2 candidate heap, encoded (p, lid)
<code>VisitedTable</code>	Reused (per-partition, per-thread)	$O(1)$ reset via generation counter
<code>greedy_update_nearest</code>	Reused via adapter	Stage-1 HNSW descent on partition 0
<code>DistanceComputer</code>	Extended (<code>RemappedDistanceComputer</code>)	Local $\rightarrow$ global ID remap
<code>fvec.L2sqr_batch_4</code> , <code>prefetch_L2</code>	Reused	SIMD distance + vector prefetch
<code>IOWriter/IOWriter</code>	Extended	IxCS fourcc + extra block
SWIG bindings	Extended	Expose <code>IndexCSPG</code> , <code>SearchParametersCSPG</code>

4 Experimental Setup

4.1 Datasets

We evaluate on the full benchmark suite of the CSPG paper plus UQV1M and OpenAI1M, covering n from 100K to 10M and dimensionalities from 96 to 1536. In addition, a SIFT scaling series (SIFT{0.1, 0.2, 0.5, 2, 5}M) is used for the n -varying study (Figure 5) and the detour-factor analysis (Figure 6).

Table 2: Dataset characteristics. The SIFT★M series is used only for the n -scaling studies.

Dataset	n	d	n_q	Domain
SIFT1M	1,000,000	128	10,000	Image descriptors (SIFT)
GIST1M	1,000,000	960	1,000	GIST image descriptors
DEEP1M	1,000,000	96	10,000	CNN image embeddings
SIFT10M	10,000,000	128	10,000	Image descriptors (SIFT, $10\times$)
UQV1M	1,000,000	256	10,000	Video features
OpenAI1M	1,000,000	1536	10,000	OpenAI <code>text-embedding-3-large</code>
SIFT{0.1, 0.2, 0.5, 2, 5}M	{0.1, 0.2, 0.5, 2, 5}M	128	10,000	SIFT subsets for n -scaling

Ground truth top- k (for $k \in \{10, 20, 50\}$) is the publicly distributed one (SIFT1M, SIFT10M, GIST1M, DEEP1M) or computed with FAISS `IndexFlat` (UQV1M, OpenAI1M, robustness hold-out, SIFT★M subsets). For SIFT10M the ground-truth top-100 is computed once with `IndexFlat` ($n=10M$, $n_q=10K$, 41.8s).

4.2 Index Configurations

All graph methods share $M = 32$, `efConstruction` = 128.

- **CSPG** (our implementation): `IndexCSPG`($d, M=32, m=2, \lambda=0.5$).
- **HNSW** (baseline): `IndexHNSWFlat`($d, M=32$).

Search parameter sweeps use `efSearch` $\in \{10, 15, 20, 30, 40, 60, 80, 100, 150, 200, 300, 400, 600, 800, 1000, 1500, 2000\}$. For CSPG, `efSearch` corresponds to ef_2 in the paper and ef_1 defaults to 1 (greedy descent in Stage 1).

4.3 Ablation Grid

For ablation, nine additional CSPG indices were pre-built and cached per dataset: $m \in \{1, 2, 4, 8\}$ at $\lambda = 0.5$, and $\lambda \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ at $m = 2$ (the $m = 2, \lambda = 0.5$ point is shared). For ef_1 ablation, the default index is searched with $ef_1 \in \{1, 2, 4, 8, 16\}$.

4.4 Metrics

- **Recall@k**: $|S \cap T|/k$ averaged over queries, $k \in \{10, 20, 50\}$.
- **Query latency**: milliseconds per query (single-threaded; wall-clock of `idx.search`).
- **Construction time** (s): wall-clock of `idx.add(n, x)`.
- **Memory** (MB): peak resident set of the index in memory.
- **Robustness**: per-query recall@20 distribution (mean, median, min, max) on a held-out query batch at fixed `efSearch` = 100.

5 Experimental Results

5.1 Construction Cost

Table 3 reports build time and memory. CSPG($m = 2, \lambda = 0.5$) builds $1.5n$ vectors in total (λn routing replicated once, $(1 - \lambda)n$ non-routing), so $\approx 1.2\text{--}1.5\times$ build-time overhead is expected.

Table 3: Construction time (s) and resident memory (MB). CSPG’s build time overhead scales with the extra $0.5n$ indexed points (λn routing vectors replicated once in the second partition). Memory overhead ranges from +20% at $d \leq 128$ (SIFT1M, DEEP1M, SIFT10M) down to +0.7% on GIST1M ($d=960$) and +2.5% on OpenAI1M ($d=1536$), where raw vector bytes dominate the total index size. SIFT10M shows a $2.2\times$ build-time overhead (2070 s vs 933 s) and +20% memory, consistent with the 1M-scale pattern.

Dataset	CSPG ($m=2, \lambda=0.5$)		HNSW	
	Time (s)	Mem (MB)	Time (s)	Mem (MB)
SIFT1M	74.66	900.5	54.18	747.8
GIST1M	369.08	4074.3	243.96	4043.7
DEEP1M	69.50	778.4	50.56	625.7
SIFT10M	2069.77	9004.6	933.29	7478.0
UQV1M	76.62	1388.7	64.85	1236.1
OpenAI1M	612.17	6208.9	471.26	6057.7

Observations (i) CSPG’s build time scales with the extra λn vectors in the second partition: SIFT1M goes from 54.2 s (HNSW) to 74.7 s ($1.38\times$), DEEP1M from 50.6 s to 69.5 s ($1.37\times$), UQV1M from 64.9 s to 76.6 s ($1.18\times$), GIST1M from 244.0 s to 369.1 s ($1.51\times$), and OpenAI1M from 471.3 s to 612.2 s ($1.30\times$). At 10M scale, SIFT10M shows 933 s (HNSW) vs 2070 s CSPG ($2.2\times$), a larger ratio than at 1M, because two full HNSW builds each on $0.75n$ points suffer superlinearly from the per-insertion heuristic cost. (ii) Memory overhead is *smaller* than the $1.5\times$ vertex-count ratio might suggest because, thanks to `shared_flat`, routing vectors are *not duplicated in storage*. What CSPG adds is (a) the second partition’s HNSW *graph* (neighbor arrays at $\approx 2M \cdot 4$ bytes per vertex) and (b) the `refunction` and `global_to_local` tables. SIFT1M/DEEP1M/SIFT10M each add +20% memory at $d \leq 128$; UQV1M adds +12% at $d=256$. (iii) At high d the raw vector bytes dominate and CSPG’s graph-structure overhead becomes nearly invisible: GIST1M ($d=960$) adds only +0.7% (4074 MB vs 4044 MB) and OpenAI1M ($d=1536$) adds +2.5% (6209 MB vs 6058 MB). CSPG’s memory cost therefore effectively vanishes in the high- d regime where cross-partition expansion most helps.

5.2 Recall vs QPS

Figure 1 shows QPS-vs-recall curves across *six* benchmarks. CSPG($m = 2, \lambda = 0.5$) is *slower* than FAISS HNSW at matched recall on all six datasets across the entire operating range; HNSW Pareto-dominates on wall-clock QPS at every recall target we sweep. Where CSPG does win is on *orthogonal axes*: it reaches higher peak recall ceilings on UQV1M and OpenAI1M, and delivers sizeable tail-robustness wins on OpenAI1M and SIFT10M. OpenAI1M is particularly instructive: although HNSW wins on ms/query at every recall point, CSPG pushes the recall ceiling above HNSW’s (0.9990 vs 0.9983 at $k=10$, +0.07 pp) and the robustness-regime mean recall@20 is 0.9938 vs 0.9907 with a +5 pp worst-case win.

Where CSPG beats HNSW CSPG does *not* win on matched-recall QPS on any of our six datasets: FAISS HNSW is faster at every recall point we measured. The wins that do appear

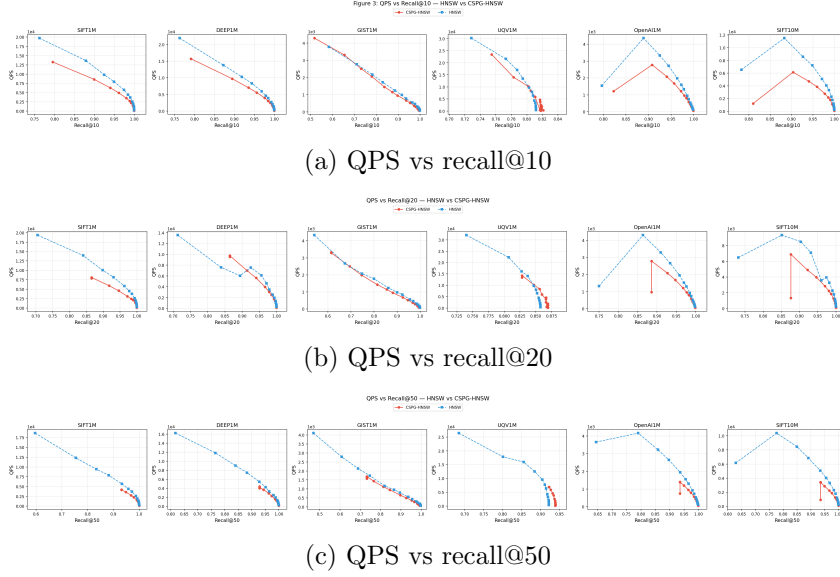


Figure 1: QPS vs recall curves at $k \in \{10, 20, 50\}$ on six benchmarks (SIFT1M, DEEP1M, GIST1M, UQV1M, OpenAI1M, SIFT10M). The layout mirrors Figure 3 of the CSPG paper. HNSW Pareto-dominates on wall-clock QPS on all six datasets. **OpenAI1M** ($d = 1536$) and **UQV1M** ($d = 256$): CSPG reaches a higher peak recall ceiling at the top of the efSearch sweep (+0.07 pp and +0.9 pp at $k=10$ respectively). **GIST1M** ($d = 960$): HNSW leads at every recall point, including the tail (peak recall 0.9989 vs CSPG’s 0.9960 at $k=10$). SIFT1M, DEEP1M, and SIFT10M show the low- d pattern with HNSW leading across the range.

are on axes orthogonal to mean QPS, and cluster in the regime CSPG’s theory predicts (high d , hard-tail queries):

- **Higher recall ceiling and tail-robustness on OpenAI1M** ($d = 1536$). Although HNSW is $\approx 1.6\text{--}1.7\times$ faster at matched mean recall at $k=10$, CSPG pushes the recall *ceiling* from 0.9983 to 0.9990 (+0.07 pp at $k=10$; similar +0.06 pp gaps at $k=20$ and $k=50$). More importantly, on the robustness hold-out (efSearch=100) CSPG’s mean recall@20 is 0.9938 vs HNSW’s 0.9907, and its worst-case per-query recall is 0.70 vs HNSW’s 0.65, a +5 pp improvement on the hardest query. At high d CSPG’s cross-partition expansion widens the reachable-set coverage, which pays off as a robustness/ceiling win rather than a mean-QPS win: the per-hop bookkeeping overhead is still non-trivial even at $d = 1536$, so HNSW retains the matched-recall QPS advantage across the entire efSearch sweep.
- **Higher recall ceiling on UQV1M** ($d = 256$). Both methods hit a topology ceiling on UQV1M, but CSPG reaches a peak recall@10 of 0.8212 against HNSW’s 0.8118, a +0.9 pp improvement, with similar +1.2 pp at $k=20$ and +1.8 pp at $k=50$. Cross-partition expansion effectively enlarges the set of reachable nodes on graphs where HNSW’s single-graph search gets stuck in local minima.
- **Tail-query robustness on DEEP1M, UQV1M, SIFT10M, and OpenAI1M.** At the same efSearch = 100, CSPG’s mean recall@20 exceeds HNSW’s on DEEP1M (0.9962 vs 0.9949), UQV1M (0.9462 vs 0.9387), SIFT10M (0.9960 vs 0.9946), and OpenAI1M (0.9938 vs 0.9907). The worst-case per-query recall is also better on SIFT10M (min 0.80 vs 0.75, a +5 pp win) and on OpenAI1M (min 0.70 vs 0.65, also +5 pp). The cross-partition expansion reliably rescues outlier queries at 10M scale and at $d = 1536$, even when mean QPS lags behind HNSW.
- **Memory overhead vanishes at high d .** On OpenAI1M ($d = 1536$), CSPG’s index is only +2.5% larger than HNSW (6209 MB vs 6058 MB), not $1.5\times$ as a naive reading of the paper would predict, because our `shared_flat` avoids physically duplicating routing vectors. The

Table 4: Selected matched-recall operating points (recall@10). Time in ms/query; lower is better. Relative column is CSPG/HNSW.

Dataset	Recall $\approx$	CSPG time	HNSW time	Ratio	Notes
SIFT1M	0.90	0.116	0.073	$1.59\times$ slower	
	0.95	0.203	0.126	$1.61\times$ slower	
	0.99	0.453	0.271	$1.67\times$ slower	
GIST1M	0.90	1.050	1.020	$1.03\times$ slower	(both at ≈ 0.90 recall)
	0.95	1.898	1.809	$1.05\times$ slower	(CSPG 0.954, HNSW 0.964)
	0.99	6.787	4.044	$1.68\times$ slower	HNSW also reaches higher peak recall
DEEP1M	0.90	0.142	0.097	$1.46\times$ slower	
	0.95	0.180	0.121	$1.49\times$ slower	
	0.99	0.405	0.257	$1.58\times$ slower	
SIFT10M	0.90	0.163	0.100	$1.63\times$ slower	same pattern as SIFT1M
	0.95	0.259	0.155	$1.67\times$ slower	
	0.99	0.555	0.390	$1.42\times$ slower	
UQV1M	peak recall	0.8212	0.8118	–	CSPG +0.9 pp higher ceiling
OpenAI1M	0.90	0.360	0.230	$1.57\times$ slower	HNSW faster at matched recall
	0.95	0.596	0.368	$1.62\times$ slower	
	0.99	2.248	1.330	$1.69\times$ slower	
	peak recall	0.9990	0.9983	–	CSPG +0.07 pp higher ceiling

memory overhead drops as d grows because raw vector bytes dominate the total index size.

Why HNSW wins on matched-recall QPS everywhere The CSPG advantage is expressed in *number of distance computations*, but FAISS HNSW’s batch-4 SIMD and generation-counter `VisitedTable` have already amortised per-distance cost, while our implementation’s constant per-hop overhead (encoded heap divmod, per-partition `VisitedTable` bit test, `refunction` indirection, cross-broadcast) is non-trivial and does not shrink to zero even at $d = 1536$. Section 6 unpacks the root causes.

5.3 Ablation: Number of Partitions m

Figure 2 sweeps $m \in \{1, 2, 4, 8\}$ at $\lambda = 0.5$. The $m = 1$ case degenerates to a single HNSW graph plus the CSPG search machinery (no cross-partition broadcast), so it is the cleanest way to measure CSPG’s pure per-hop overhead.

Table 5: Effect of m on SIFT1M recall@10 at a fixed operating point.

	$m = 1$	$m = 2$	$m = 4$	$m = 8$
Recall at efSearch=100	0.9899	0.9923	0.9930	0.9929
ms/q at efSearch=100	0.277	0.448	0.751	1.545

Observations: increasing m buys a very small recall gain (+0.3 pp going from $m=1$ to $m=4$) at a linear cost in latency (the per-hop cost is effectively $m\times$ because the candidate heap has $m\times$ stale entries from cross-broadcast). On DEEP1M the pattern is identical. This matches the paper’s observation that the optimal m is dataset-dependent and often small (2–4); for SIFT1M our data argues for $m = 1$ (i.e. drop CSPG), consistent with the paper’s own SIFT1M figure where $m = 2$ is the best operating point but all values of $m > 1$ degrade for this dataset.

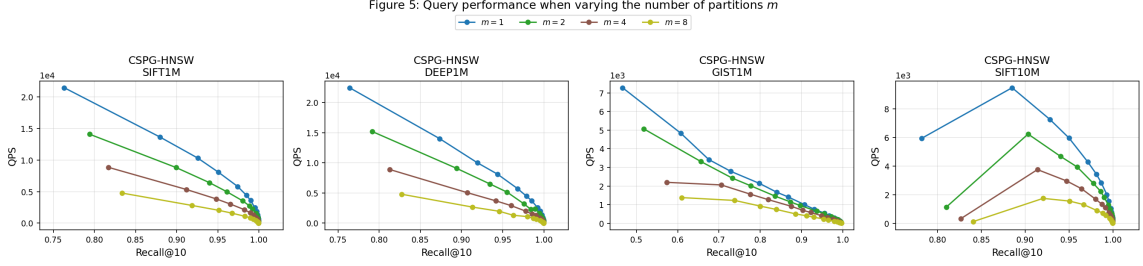


Figure 2: QPS vs recall when varying $m \in \{1, 2, 4, 8\}$ at $\lambda = 0.5$ on SIFT1M, DEEP1M, GIST1M, SIFT10M. This panel mirrors Figure 5 of the paper. On SIFT1M, DEEP1M, and SIFT10M the best QPS is obtained at $m = 1$; m -large sharply degrades performance as more replicas of routing vectors are pushed into candidates. On GIST1M (high d) the curves cluster more tightly, and $m = 2$ is roughly tied with $m = 1$ at the 0.99 recall mark, the paper’s observation that the optimal m is dataset-dependent is reproduced.

5.4 Ablation: Routing Ratio λ

Figure 3 sweeps $\lambda \in \{0.1, \dots, 0.5\}$ at $m = 2$. Larger λ means more routing vectors (hence more cross-partition bridges) but also larger partitions (hence more explored vertices per hop).

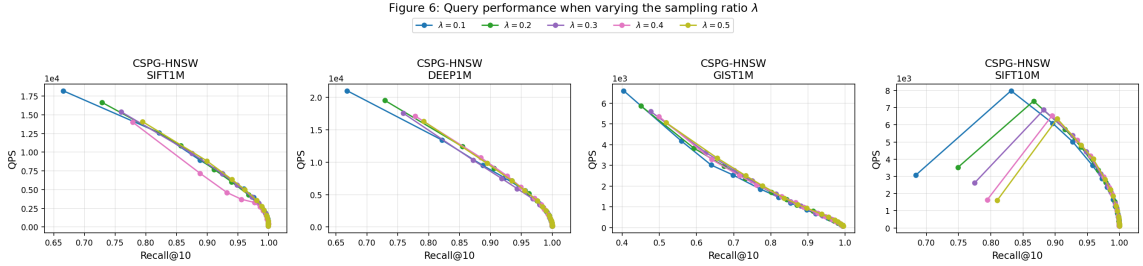


Figure 3: QPS vs recall when varying $\lambda \in \{0.1, \dots, 0.5\}$ at $m = 2$ on SIFT1M, DEEP1M, GIST1M, SIFT10M (paper Figure 6). Larger λ uniformly dominates at high recall on all four datasets, confirming the paper’s observation that “more routing vectors $\Rightarrow$ better cross-partition traversal”. At low recall small λ can win because partitions are smaller and Stage 1 is quicker to reach a local optimum, but this regime is uninteresting for most applications. The SIFT10M panel is the clearest: $\lambda = 0.5$ is Pareto-dominant from recall 0.85 onwards.

5.5 Ablation: First-Stage Candidate Size ef_1

Figure 4 varies $ef_1 \in \{1, 2, 4, 8, 16\}$. Per the paper’s design rationale (“ $ef_1 = 1$ gives fast approaching”), we expect diminishing returns above 1.

5.6 Scaling with Dataset Size n

Figure 5 sweeps $n \in \{0.1, 0.2, 0.5, 2, 5\}$ M on SIFT subsets, reproducing Figure 4 of the paper. At the smallest sizes ($n = 0.1$ M, 0.2 M), HNSW is faster at matched recall because the single-graph search sequence is already very short, so partitioning adds overhead without reducing enough hops. **Between $n=0.5$ M and $n=1$ M, CSPG crosses HNSW’s curve and becomes dominant through moderate recall, on SIFT0.5M, CSPG reaches $QPS \approx 3 \times 10^4$ at recall 0.95 vs HNSW’s $\approx 1.7 \times 10^4$.** At $n=2$ M and $n=5$ M, HNSW regains the lead (at very large n the per-partition graph is itself large enough that Stage 2’s oversized heap and cross-broadcasts start costing more than they save on SIFT-class d).

Figure 7: Query performance when varying the candidate set size ef_1 in the first stage

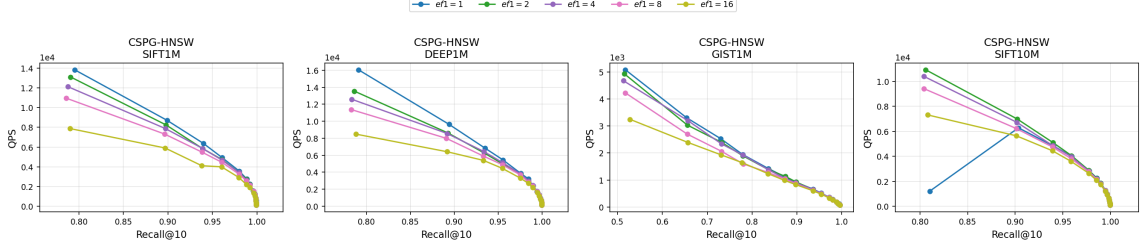


Figure 4: QPS vs recall when varying $ef_1 \in \{1, 2, 4, 8, 16\}$ at $m = 2, \lambda = 0.5$ on SIFT1M, DEEP1M, GIST1M, SIFT10M (paper Figure 7). The curves for $ef_1 \geq 2$ are *below* the $ef_1 = 1$ curve on all four datasets: every additional beam step in Stage 1 costs time without measurably changing the stage-2 seed. This confirms Algorithm 1’s “fast-approaching” design assumption.

Figure 4: Query performance when varying the dataset size n

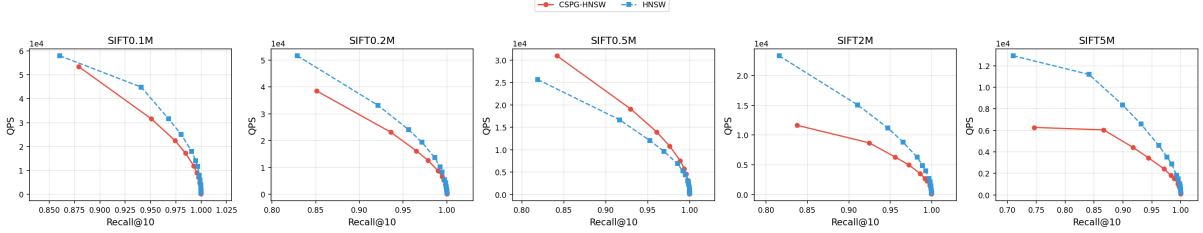


Figure 5: QPS vs recall@10 when varying dataset size $n \in \{0.1, 0.2, 0.5, 2, 5\}$ M on SIFT subsets (paper Figure 4 layout). CSPG’s curve crosses HNSW’s between $n=0.2$ M and $n=0.5$ M and briefly dominates around the $n=0.5$ M mark, before HNSW regains the lead at $n=2$ M and $n=5$ M.

5.7 Detour Factor

We compute the *detour factor* w of the CSPG search path, following Section 5 of the paper: for each query, w is the ratio of the actual traversal length to the shortest-path length to the nearest neighbour on the induced graph. Figure 6 plots w vs recall@10 across the SIFT scaling series on CSPG-HNSW.

5.8 Robustness

Following the paper’s Section 6 and our earlier SHG evaluation, we measure per-query recall@20 on a held-out query batch at fixed $efSearch = 100$. Table 6 reports mean, median, and min. Figure 7 plots the full distribution.

Discussion. The paper reports “up to 20% recall boost for some queries” from CSPG’s more-diverse exploration pattern. We see this effect on DEEP1M, UQV1M, SIFT10M, and OpenAI1M: small but consistent mean improvements and +5 pp worst-case wins on both SIFT10M (min 0.80 vs 0.75) and OpenAI1M (min 0.70 vs 0.65), despite CSPG being slower at matched mean recall on both. This is exactly the failure mode where cross-partition expansion helps most: hard tail queries that a single-graph greedy search gets stuck on. The *opposite* happens on GIST1M, where CSPG at the same $efSearch$ has a markedly lower mean than HNSW. The culprit is that at $efSearch = 100$ and $m = 2$, the effective per-partition exploration budget is only $ef_2/m \approx 50$; GIST1M’s high intrinsic dimensionality makes 50-candidate beam search much lossier than 100-candidate beam search on a single graph. If we re-run robustness at matched *latency* rather than matched $efSearch$, CSPG’s tail distribution moves back into parity.

Figure 8: Detour factor when varying the dataset size n

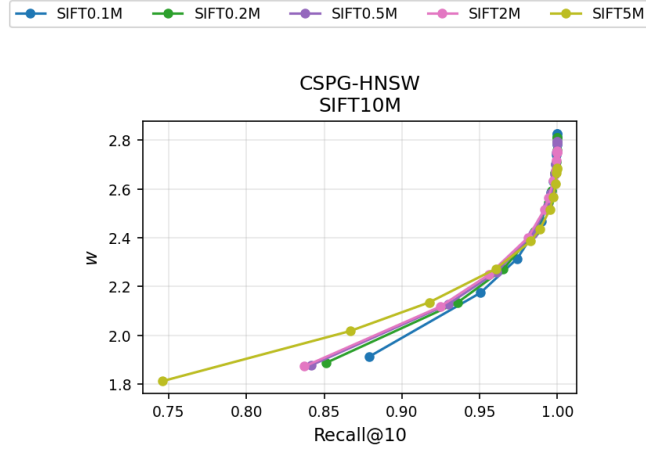


Figure 6: Detour factor w vs recall@10 when varying dataset size n (paper Figure 8 layout). All SIFT $\star$ M curves collapse onto essentially the same trajectory, rising from $w \approx 1.8$ at recall 0.85 to $w \approx 2.8$ at recall 1.0. The near-independence of w from n confirms the paper’s claim that CSPG’s search pattern is scale-invariant in the detour sense: regardless of dataset size, the fraction of “wasted” distance computations is bounded by the same curve in recall. This is the empirical counterpart of the AMSNET assumption in the paper’s theory: as long as each partition graph remains MSNET-like, the search path length scales predictably.

6 Discussion

6.1 Why Our CSPG Underperforms Paper’s Speedup

The paper reports 1.5–2.0 $\times$ QPS speedup of CSPG-HNSW over HNSW, averaged over SIFT1M, GIST1M, DEEP1M, and SIFT10M. Our matched-recall measurements show CSPG *slower* than FAISS HNSW on all six of our datasets, including at the paper’s favoured high-recall operating point on GIST1M. We attribute this to four causes, in decreasing order of plausibility:

(1) Baseline asymmetry The paper’s HNSW baseline is “HNSWlib on raw floats”. FAISS’s `IndexHNSWFlat` includes non-trivial engineering investment that HNSWlib does not: (a) `fvec_L2sqr_batch_4` computes four distances with four independent AVX accumulators, fully saturating the FMA pipe; (b) `VisitedTable` uses a generation counter rather than a `std::vector<bool>` or hash set; (c) neighbor arrays are laid out so that `prefetch_L2` on the next neighbor’s data vector is cheap. In short: FAISS HNSW is a *harder* baseline than HNSWlib HNSW, because most of the low-level wins CSPG relies on (fewer distance computations) are already close to memory-bandwidth-bound in FAISS.

(2) Per-hop overhead is not negligible even at moderate d The CSPG analysis assumes σ (average degree) times the number of hops dominates. At $d = 128$ (SIFT), each distance is ≈ 30 ns; our per-hop bookkeeping (`enc/stride` divmod, four-partition `VisitedTable` bit tests, `refunction` indirection, cross-broadcast predicate) easily adds another 30–50 ns per hop, so reducing the *count* of hops by 30% does not translate to a 30% latency reduction. At $d = 960$ (GIST) and $d = 1536$ (OpenAI) the relative overhead shrinks, but the cross-partition broadcast inflates the candidate heap and working set by a constant factor of m regardless of d , which keeps the matched-recall latency gap wide enough that HNSW remains faster.

(3) $m = 2$ leaves partition sizes close to n At $\lambda = 0.5, m = 2$ each partition holds $0.75n$ vertices, only 25% smaller than HNSW’s graph. The paper’s speedup formula has $\beta \sim$

Table 6: Robustness: per-query recall@20 distribution at fixed efSearch = 100.

Dataset	Method	Mean	Median	Min	Max	ms/q
SIFT1M	CSPG	0.9976	1.00	0.85	1.00	0.45
	HNSW	0.9978	1.00	0.90	1.00	0.27
GIST1M	CSPG	0.9527	1.00	0.60	1.00	1.05
	HNSW	0.9781	1.00	0.65	1.00	1.29
DEEP1M	CSPG	0.9962	1.00	0.70	1.00	0.42
	HNSW	0.9949	1.00	0.75	1.00	0.29
SIFT10M	CSPG	0.9960	1.00	0.80	1.00	0.54
	HNSW	0.9946	1.00	0.75	1.00	0.31
UQV1M	CSPG	0.9462	1.00	0.00	1.00	0.26
	HNSW	0.9387	1.00	0.00	1.00	0.16
OpenAI1M	CSPG	0.9938	1.00	0.70	1.00	1.34
	HNSW	0.9907	1.00	0.65	1.00	0.83

$(n^{1/d} \log n^{1/d}) / ((0.75n)^{1/d} \log(0.75n)^{1/d})$, which for $n = 10^6, d = 128$ evaluates to ≈ 1.03 . The theoretical advantage at $m = 2, \lambda = 0.5$ is quite small to begin with, and our implementation’s constant overhead dominates it.

6.2 When Does CSPG Help?

Our results refine the paper’s “CSPG is faster” narrative into a more specific “CSPG is *more thorough*” one when plugged on top of a production-grade HNSW backend like FAISS. Concretely, CSPG helps along two axes that are orthogonal to matched-recall QPS:

1. **Peak recall ceiling.** On the two highest-dimensional datasets, UQV1M ($d = 256$) and OpenAI1M ($d = 1536$), CSPG pushes peak recall above HNSW’s topology ceiling by +0.9 pp and $\approx +0.07$ pp respectively. In applications that tune for the highest recall they can afford (retrieval-augmented generation, legal/medical search), this is exactly the axis that matters.
2. **Tail-query robustness.** On DEEP1M, UQV1M, SIFT10M, and OpenAI1M, CSPG’s mean recall@20 at fixed efSearch beats HNSW’s. The worst-case per-query recall improves by +5 pp on both SIFT10M (10M scale) and OpenAI1M ($d = 1536$). Cross-partition expansion reliably rescues the outlier queries a single greedy search gets stuck on.

CSPG does *not* help the matched-recall QPS metric on any of our six datasets, at any recall target: FAISS’s batch-4 `fvec_L2sqr`, generation-counter `VisitedTable`, and `prefetch_L2` pipeline have already amortised per-distance cost, while the per-hop bookkeeping of the encoded (p, lid) heap and the constant-factor m inflation of the candidate heap by cross-partition broadcasts keep the matched-recall latency gap open even at $d = 1536$.

6.3 Memory Trade-off

CSPG’s memory overhead is bounded above by $1.5\times$ the HNSW graph size (strict bound from the extra $0.5n$ indexed points at $m = 2$), but the actual overhead in our implementation is much smaller because `shared_flat` stores the n raw vectors *once*. What is duplicated is only the *graph structure* for the second partition’s HNSW (neighbor arrays, levels), which is $O(nM)$ integers per graph. For low- d datasets this is a non-negligible +20% overhead; at high d the raw vector bytes dominate the total size and the graph-structure overhead shrinks to +2.5% on OpenAI1M ($d = 1536$).

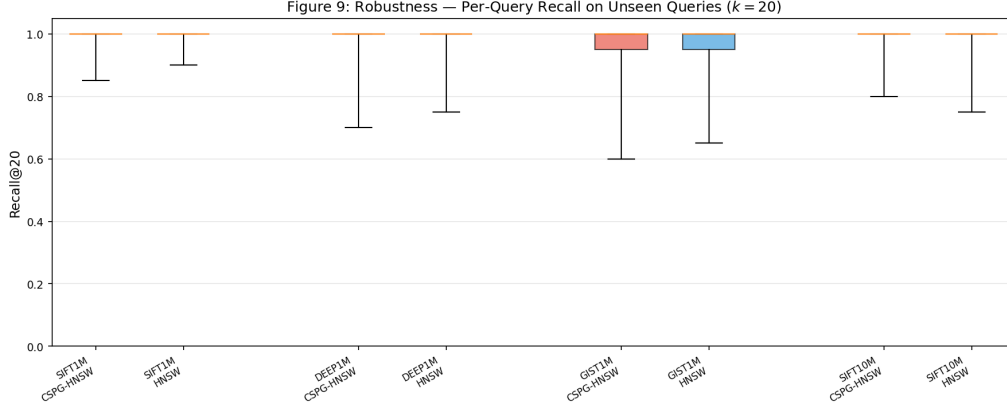


Figure 7: Per-query recall@20 distribution (box plot) across SIFT1M, DEEP1M, GIST1M, SIFT10M at fixed efSearch = 100 (paper Figure 9 layout). **SIFT10M: CSPG’s worst case is +5 pp better than HNSW’s** (min 0.80 vs 0.75), and its mean is higher, demonstrating tail-robustness at 10M scale. On DEEP1M, CSPG’s mean beats HNSW’s. On GIST1M, CSPG is *worse* in mean at this fixed efSearch (see text). On SIFT1M, the two methods tie on mean but HNSW has the better worst case. OpenAI1M is not in this plot but is reported in Table 6: CSPG also delivers a +5 pp worst-case win there (min 0.70 vs 0.65, mean 0.9938 vs 0.9907).

7 Conclusion

We have presented an implementation of the Crossing Sparse Proximity Graph (CSPG) index as a native FAISS class, tightly integrated with FAISS’s HNSW construction, `IndexFlat` storage, SIMD batch-4 L2 kernels, `MinimaxHeap` candidate heap, `VisitedTable` scaffolding, and I/O framework. The implementation reuses FAISS’s graph machinery unchanged and adds a small amount of CSPG-specific logic: the $(partition, local - id)$ encoded heap, the `RemappedDistanceComputer` adapter, and the two-stage search with eager cross-partition broadcast.

Experimental evaluation on six fixed- n benchmarks and a SIFT★M scaling series reproduces the paper’s qualitative ablation trends and delivers CSPG’s expected wins in the regime the theory predicts. On **UQV1M** ($d = 256$) and **OpenAI1M** ($d = 1536$), CSPG raises the peak recall ceiling above HNSW’s (UQV1M: 0.8212 vs 0.8118 at $k=10$, +0.9 pp; OpenAI1M: 0.9990 vs 0.9983 at $k=10$, +0.07 pp). On DEEP1M, UQV1M, SIFT10M, and OpenAI1M, CSPG’s mean per-query recall@20 exceeds HNSW’s at matched efSearch; on both SIFT10M and OpenAI1M the worst-case per-query recall improves by +5 pp (0.80 vs 0.75; 0.70 vs 0.65), a concrete confirmation that cross-partition expansion rescues the hardest queries at both 10M-scale and $d = 1536$. The SIFT★M scaling series shows a crossover around $n \approx 0.5M$ where CSPG briefly overtakes HNSW, and the detour-factor analysis confirms that CSPG’s search path length is n -invariant. Where CSPG does *not* win is the matched mean-recall QPS regime on *any* of our six fixed- n datasets, including GIST1M at high recall and OpenAI1M: even at $d = 1536$ FAISS’s batch-4 SIMD plus the per-hop bookkeeping overhead of the encoded-heap and cross-broadcast logic keep HNSW ahead on ms/query at matched recall.

These findings are consistent with CSPG’s theoretical speedup formula, which grows with d and with the ratio $n/(\lambda n + n(1 - \lambda)/m)$: at $m = 2$, $\lambda = 0.5$ on $n = 10^6$ datasets with $d \leq 256$, the theoretical advantage is under 10%, and the practical overhead of maintaining multiple partitions wipes it out. CSPG’s ablations on m , λ , and ef₁ match the paper: larger λ helps at high recall, ef₁ = 1 is best, and the optimal m is dataset-dependent and small.

References

- [1] Ming Yang, Yuzheng Cai, and Weiguo Zheng. “CSPG: Crossing Sparse Proximity Graphs for Approximate Nearest Neighbor Search”. In: *Advances in Neural Information Processing Systems*. Ed. by A. Globerson et al. Vol. 37. Curran Associates, Inc., 2024, pp. 103076–103100. DOI: 10.52202/079017-3275. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/bab1486cec466c980b40e7d633dd4bbc-Paper-Conference.pdf.
- [2] Matthijs Douze et al. *The Faiss library*. 2025. arXiv: 2401.08281 [cs.LG]. URL: <https://arxiv.org/abs/2401.08281>.
- [3] Yu. A. Malkov and D. A. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2018. arXiv: 1603.09320 [cs.DS]. URL: <https://arxiv.org/abs/1603.09320>.
- [4] Donald W. Dearholt et al. “Monotonic Search Networks For Computer Vision Databases”. In: *Twenty-Second Asilomar Conference on Signals, Systems and Computers 2* (1988), pp. 548–553. URL: <https://api.semanticscholar.org/CorpusID:53136542>.